Three of the key classes explained:

Sidenote: We ultimately decided to design the classes according to MVC (Model View Controller) design framework in order to make sure that the functionalities are delegated somewhat evenly and logically between different classes (e.g. Models to store/ make a skeleton for the data (Tile, GodPower, etc.), View to manage the UI (GameFrame, TileButton), and Controller to manage the game logic and manipulate the data in Models).

1. GameController

   GameController is a Controller-level class that contains all the core logic of the game (defines movement, turn order, building, and worker selection mechanics, and validates them according to the rules, which are, technically, also defined by this class). It could have been a large method inside Main/StartingScreen, but that would go against the task delegation principle of OOP, and turn them into some kind of GodClass.

2. GameFrame

   GameFrame is a View-level class and extends JFrame. It inherits from JFrame to handle connecting the data to the actual view during initialization and subsequently update the view at the end of the turn when actions have been taken. It stores the tile buttons, labels and gameboard background image. This all could have been done in the starting screen class or game controller but this would have violated the OOP design principle and made the code very messy.

3. Player

   Player is a Model-level class that stores the data regarding the players. The interesting part about it is that it only requires a colour as a variable to set it up, and uses injectors to allow the players be able to choose their god powers. It could not have been altered/changed into a method since its only reason to exist is to define a structure rather than providing a behaviour.

Inheritance:

- Interfaces: We used interfaces with Controllers and GodPowers, even though the reasons for either were different. For the controllers, the only reason the respective interface exists is that it saves time making a pseudo-constructor for both GameFrame and TileButton to assign an event listener. In the case of GodPowers, on the other hand, the rationale behind using an interface was to make a Strategy pattern to define a set of behaviours common amongst all the god cards (so they are easier to call from any of the controllers, and do not require manual intervention for every specific god card).
- Abstraction: We only used abstraction once for the Board class, just so we could make the game board more adjustable in regards to the dimensions and shape. It

was also one of the ways to make a Factory pattern, which can be used extensively in case we decide to add more board types in future.

Relationships:

- We decided to make the board an abstract class while providing a static method to generate a square board pattern so that it could be implemented in a more modular manner that is easy to add on to. The board contains an array of tiles (ie uses composition) as this allows the GameController, which also contain a square board object, to call upon the current tile easily.
- The GameController implements an interface Controller, which allows for multiple versions of the controller to exist. The GodPowerController also contains a reference to the core GameController with composition so that shared functions between the two game modes can be called without duplication.
- The God classes Artemis and Demeter implement the interface GodPowers. This allows for certain behaviours to come into effect when needed and depending on the specific gods power. In this case of demeter and artemis this is the move and build phases.

Design Patterns used:

- Strategy pattern
    - Explanation: GodPower is an interface with methods like beforeMove, afterMove, etc., that justify a common behaviour/strategy, and each specific god implements its own behaviour in detail. At runtime, different GodPower instances are swapped/injected to change how moves and builds work without touching the controller logic.
    - Reasoning: Makes it possible to define a God power without changing anything in the controller, or any other class, really. Makes the whole thing more neat, the logic more intuitive, and makes the code more expandable, centralised and less repetitive.

- Decorator Design pattern
    - Explanation: GodPowerController (in this case being the decorator) holds an instance of the “core” GameController and delegates almost all calls to it while also interposing some extra steps introduced by the god powers. That makes it easy to extend the behaviour of GameController dynamically, without modifying its code directly.
    - Reasoning: The idea was to make the game logic work in such a way that a couple of new game modes could be added in future, and by the time the GameController was done, it was decided to save it just so, maybe, we would add a “Santorini Classic” experience once we have more game modes to present. Then, for the DRY purposes, to reduce the repetitiveness of the code, we came up with this solution for the problem of making a game with certain modifiers.

- Factory pattern

- Explanation: Board is an abstract class with a method that makes up a new instance of itself with a method (e.g. public static Board createSquare(int size)).
- Reasoning: This approach allows us to create a game board with flexible dimensions that can adapt as needed. It meets the requirement to avoid hard-coding the board size and configuration as a fixed 5x5 two-dimensional array, enabling both the size and shape of the board to be adjustable.